

Capa de Transporte

Unidad 8:

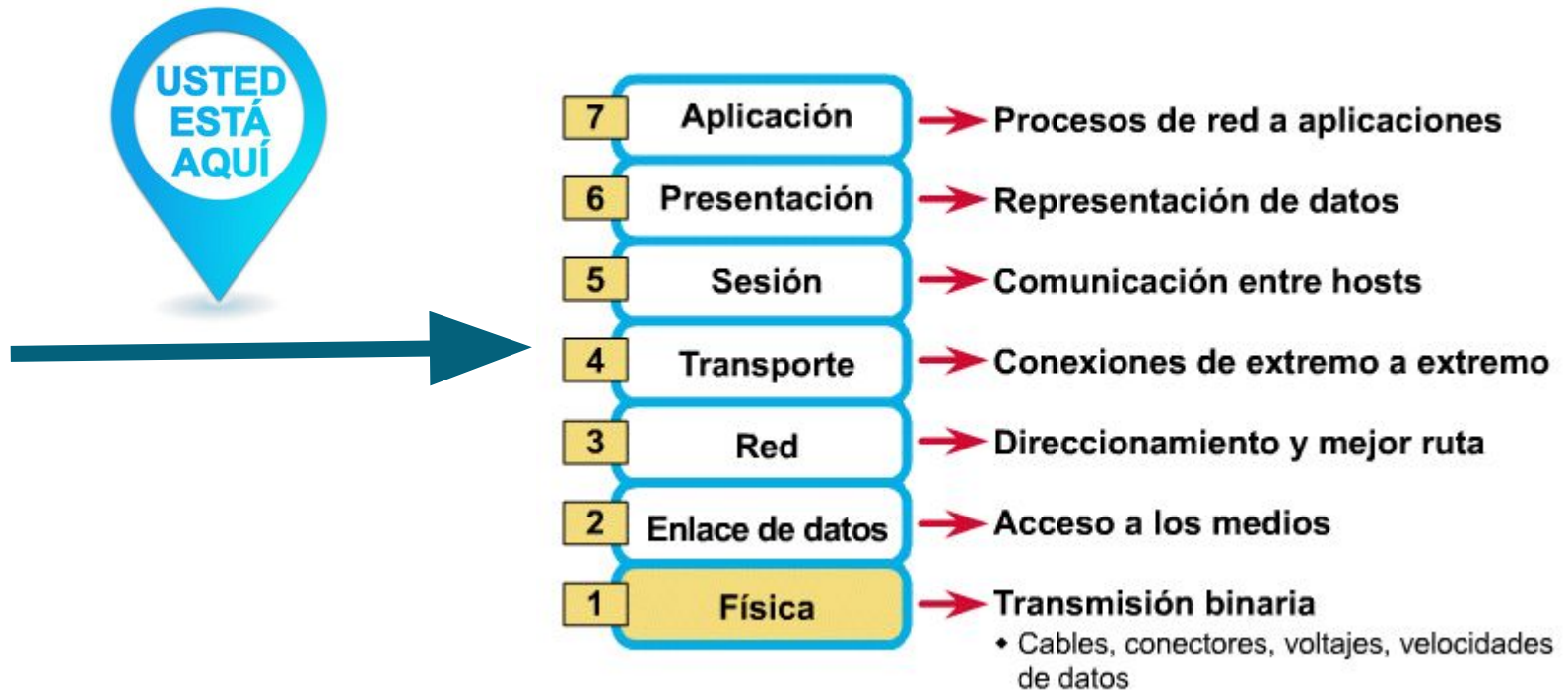
Capas de Transporte:

Protocolo TCP.

Protocolo UDP.

Capa de Transporte

Las 7 capas del modelo OSI



Servicio Confiable

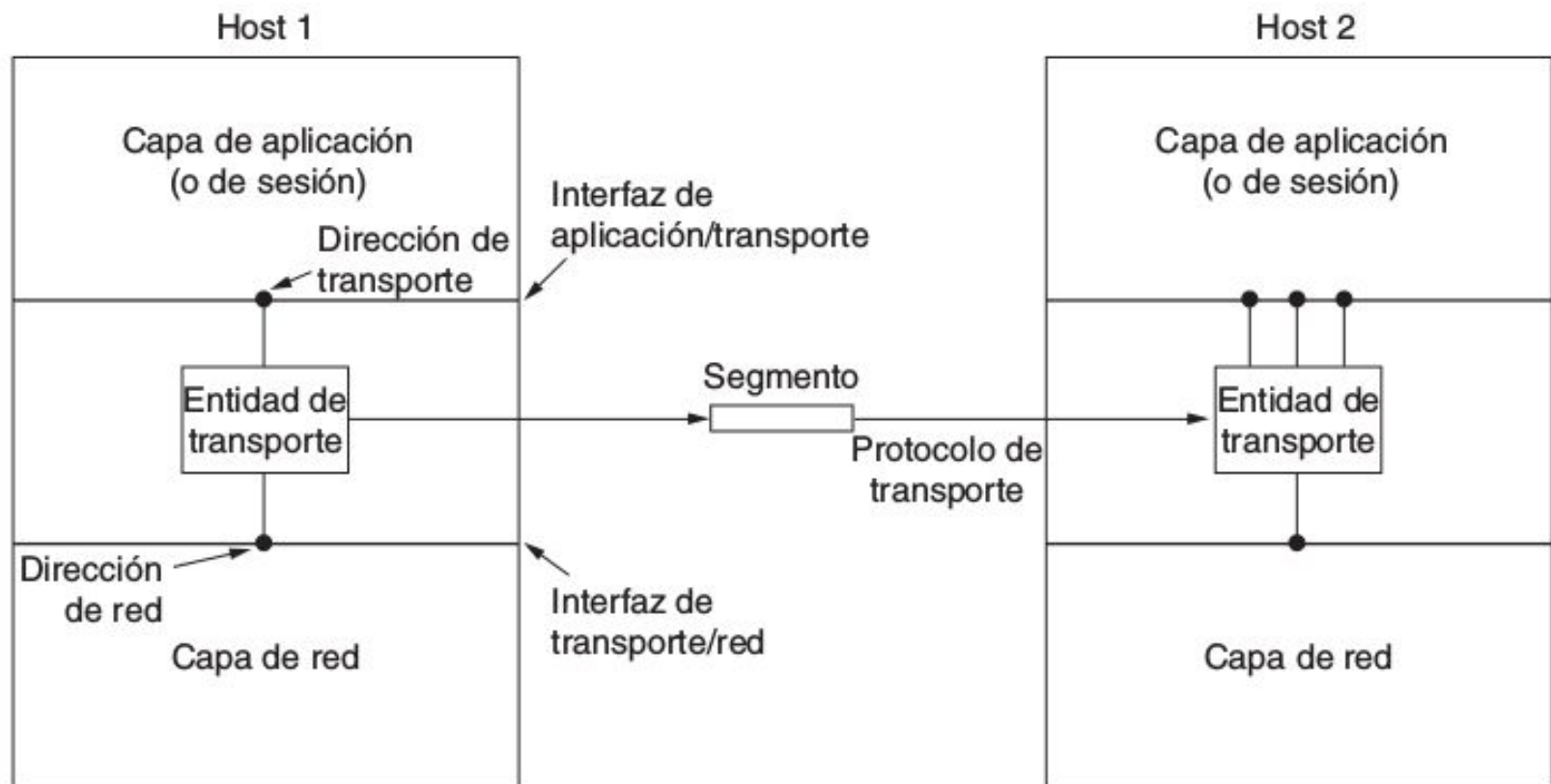
La meta fundamental de la capa de transporte es proporcionar un servicio de transmisión de datos

- **confiable** (dar la certeza/confianza de que los datos serán entregados del emisor al destino)



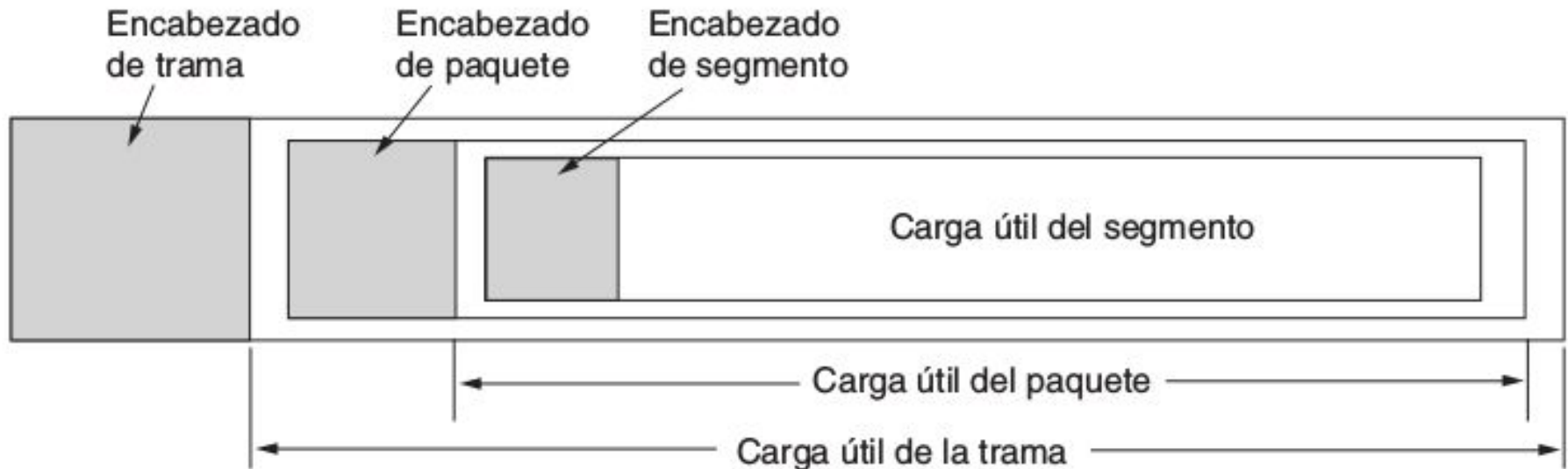
Software de la Capa de Transporte

El software de la capa de transporte que se encarga del trabajo se llama *entidad de transporte*, la cual puede localizarse en el kernel (núcleo) del sistema operativo.



Relación entre las capas de red, transporte y aplicación.

Unidad de Transporte: Segmento

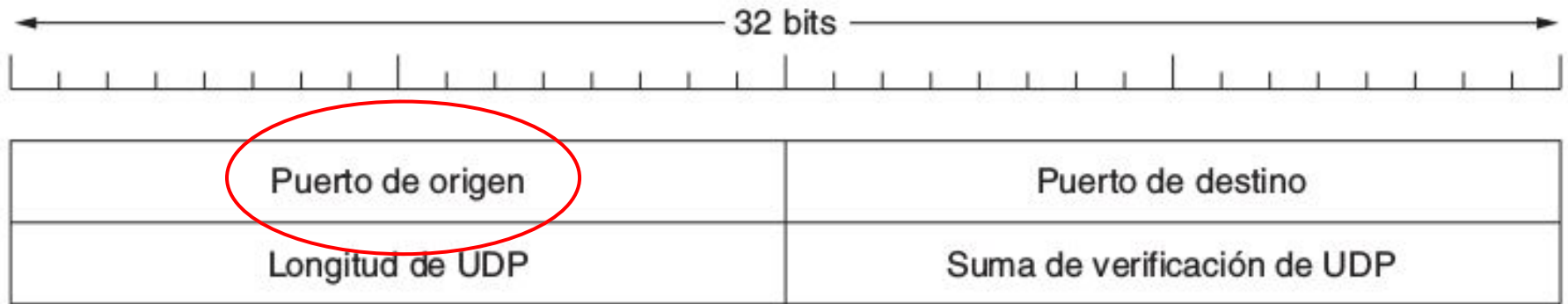


Segmento: es el mensaje que se envía de una entidad de transporte a otra.

Encapsulamiento: Cuando llega una trama, la capa de enlace de datos procesa el encabezado de la trama y, si la dirección de destino coincide para la entrega local, pasa el contenido del campo de carga útil de la trama a la entidad de red. Esta última procesa de manera similar el encabezado del paquete y después pasa el contenido de la carga útil del paquete a la entidad de transporte.

Protocolo de Transporte de Internet: UDP

(Protocolo de Datagrama de Usuario / Use Datagram Protocol)



Puertos: Son 16 bits para identificar a los procesos (servicios disponibles para la red) dentro del host.

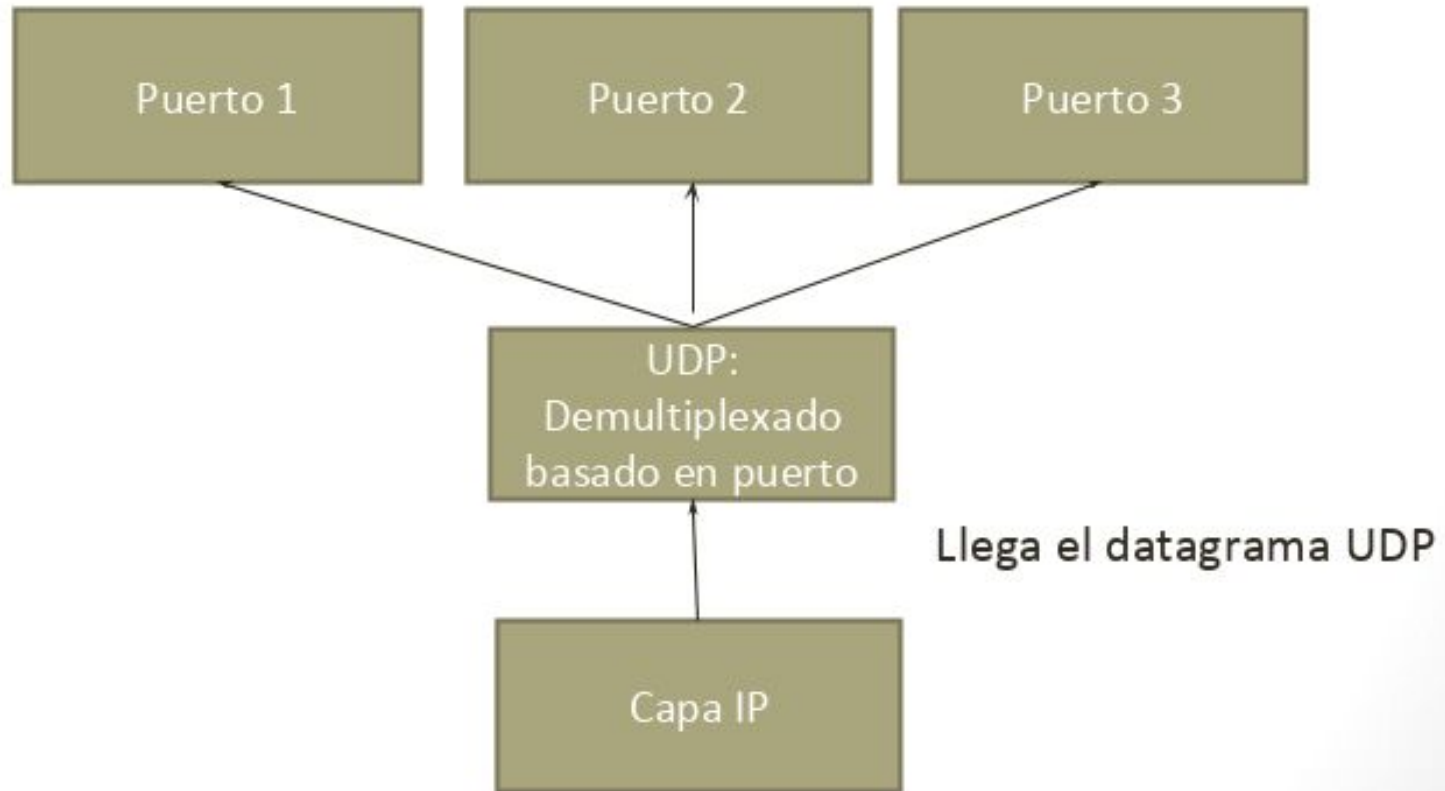
Puerto y Dirección IP componen un **Socket**

UDP transmite segmentos que consisten en un encabezado de 8 bytes seguido de la carga útil. Los dos puertos sirven para identificar los puntos terminales dentro de las máquinas de origen y destino. Cuando llega un paquete UDP, su carga útil se entrega al proceso que está conectado al puerto de destino.

Encapsulamiento UDP

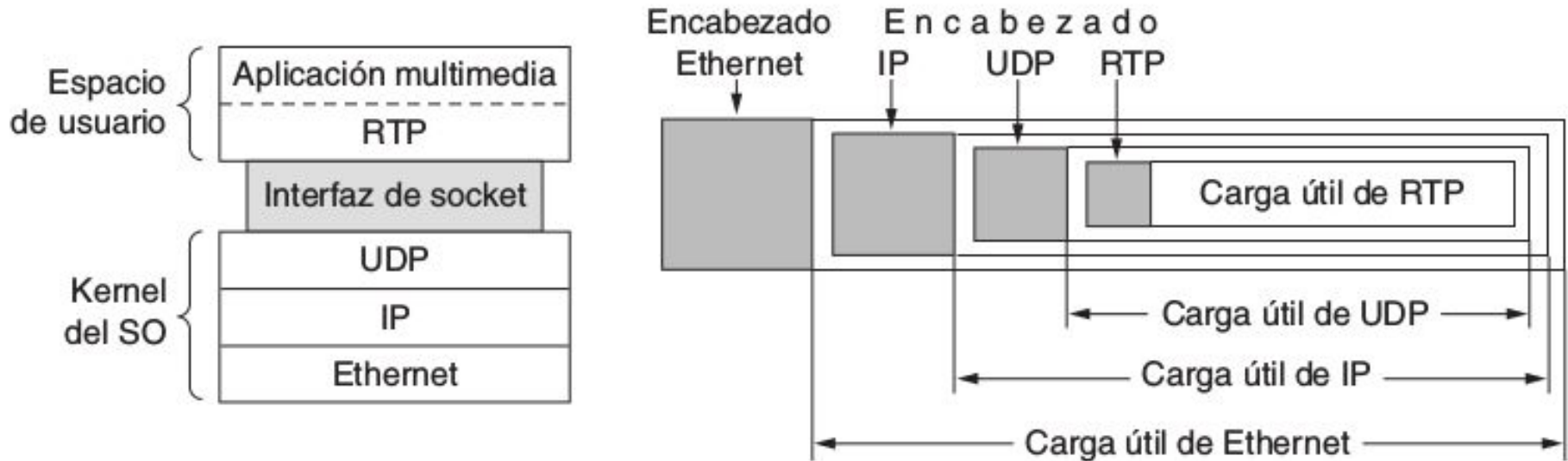


Demultiplexado UDP



Ejemplo: RTP

Protocolo de Transporte en Tiempo Real



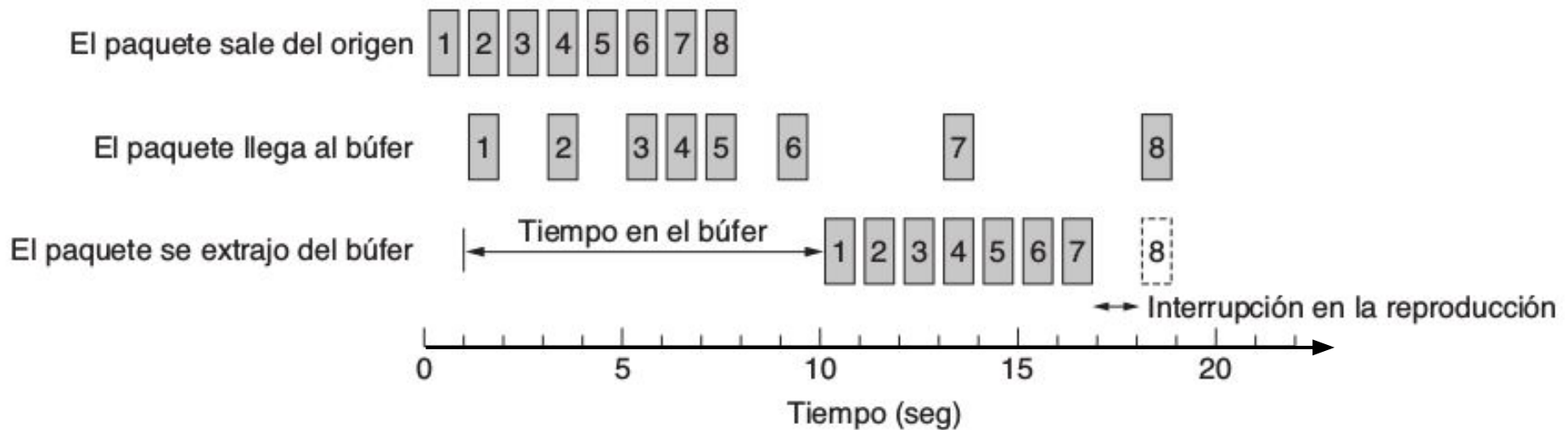
Una aplicación multimedia consiste en múltiples flujos de audio, video y texto.

Estos datos se codifican según el protocolo RTP, se envuelven en paquetes UDP y se entregan al IP para que los transmita a través de un enlace tal como Ethernet. En el receptor se lleva a cabo el proceso inverso.

Ejemplo: RTP

Protocolo de Transporte en Tiempo Real

(envío de datos multimedia, audio, video)



Para evitar el retardo de paquetes RTP (video entrecortado) se implementa un buffer en el receptor antes de reproducir.

Sockets

host X
(146.86.5.20)



web server
(161.25.19.8)



Puerto/Protocolo	Descripción
80/tcp	HTTP HTTP-ALT ver puerto 80.
22/tcp	SSH, SFTP
110/tcp	POP3 Post Office Protocol (E-mail)
3306/tcp	MySQL sistema de gestión de bases de datos

Referencia de sockets de GNU/Linux :[link1](#)

[puertos bien conocidos](#)

Ver Algunos Sockets en uso

```
sudo netstat -anp | head
```

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	127.0.0.1:17603	0.0.0.0:*	LISTEN	2692/dropbox
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	715/sshd
tcp	0	0	127.0.0.1:631	0.0.0.0:*	LISTEN	10629/cupsd
tcp	0	0	127.0.0.1:9050	0.0.0.0:*	LISTEN	720/tor
tcp	0	0	127.0.0.1:17600	0.0.0.0:*	LISTEN	2692/dropbox
tcp	0	0	192.168.1.43:46904	172.217.28.174:443	ESTABLISHED	2766/firefox-esr
tcp	0	0	192.168.1.43:44760	35.160.111.136:443	ESTABLISHED	2766/firefox-esr
tcp	1	0	192.168.1.43:58530	3.214.219.81:443	CLOSE_WAIT	2692/dropbox

Ver los puertos abiertos y escuchando TCP en mi sistema

```
sudo lsof -iTCP -sTCP:LISTEN -P -n
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
sshd	727	root	3u	IPv4	26452	0t0	TCP	*:22 (LISTEN)
sshd	727	root	4u	IPv6	26454	0t0	TCP	*:22 (LISTEN)
docker-pr	2413	root	4u	IPv6	24380	0t0	TCP	*:9000 (LISTEN)
cupsd	3254	root	9u	IPv6	31241	0t0	TCP	:::631 (LISTEN)
cupsd	3254	root	10u	IPv4	31242	0t0	TCP	127.0.0.1:631 (LISTEN)
dropbox	4342	arojas	85u	IPv4	40413	0t0	TCP	127.0.0.1:17600 (LISTEN)
dropbox	4342	arojas	101u	IPv4	40416	0t0	TCP	127.0.0.1:17603 (LISTEN)

Protocolo de Transporte de Internet: TCP

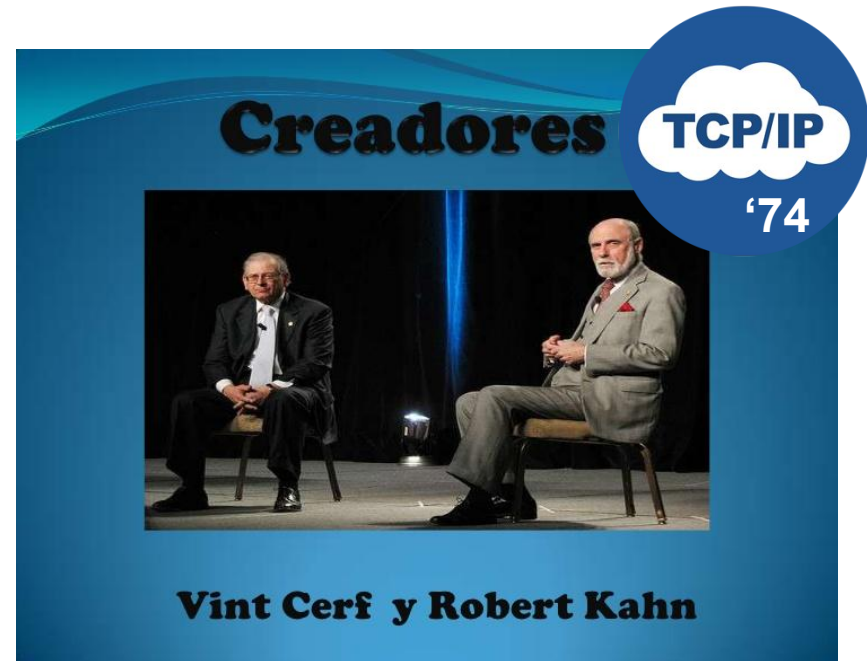
(Protocolo de / Protocol)

Protocolo principal de Internet

es orientado a conexión

Proporciona flujo de bytes

- confiable
- bidireccional
- controlado por congestión.



Encabezado Transporte TCP

Puerto de origen		Puerto de destino	
Número de secuencia			
Número de confirmación de recepción			
Long.	Sin usar		Tamaño de ventana
Suma de verificación		Apuntador urgente	

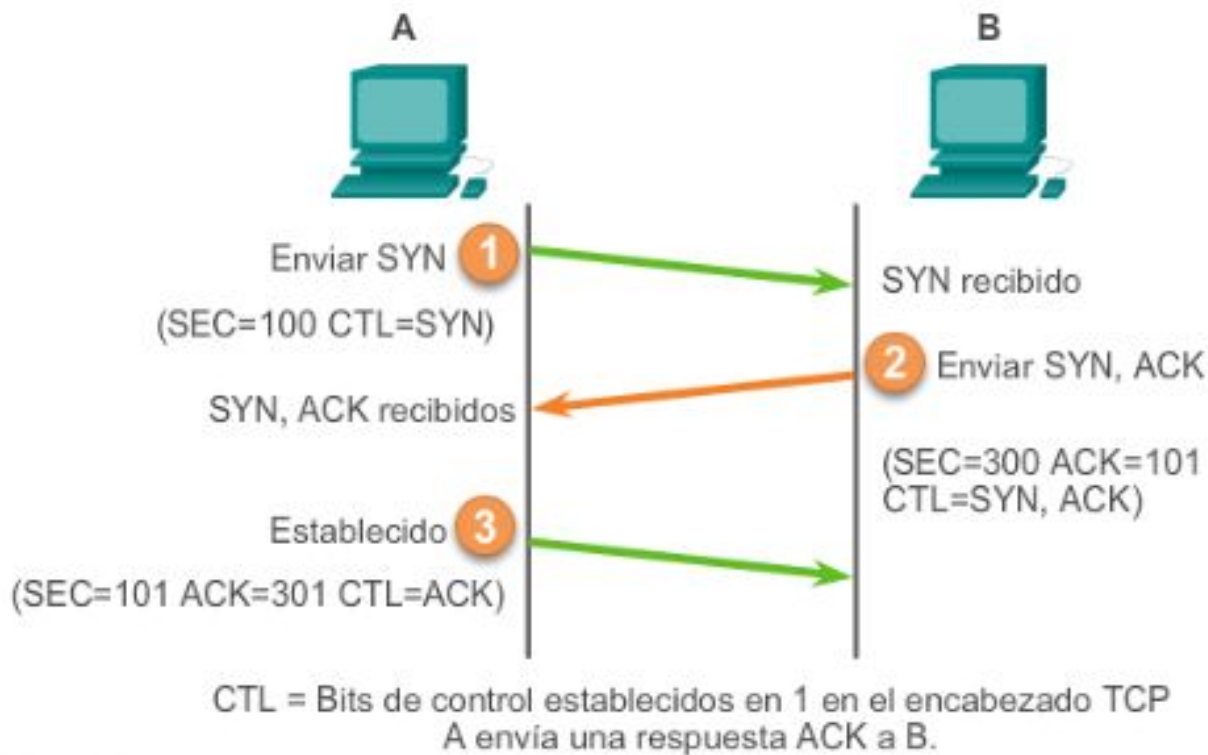
Protocolo TCP

VER.	IHL	Serv. dif.	Longitud total			
Identificación						Desplazamiento de fragmento
TTL		Protocolo		Suma de verificación del encabezado		
Dirección de origen						
Dirección de destino						

Protocolo IP

Establecimiento de una conexión

(acuerdo de 3 vías ó
three way handshake)



1 –A inicia sincronización
enviando SYN

2 –B recibe paquete y
graba sec de A y
responde con ack

3 –A indica q recibió el
ack anterior y finaliza el
proceso de conexión

Establecimiento de una conexión

(acuerdo de 3 vías ó
three way handshake)

Source	Destination	Protocol	Length	Info
192.168.0.220	82.195.132.92	TCP	66	5057→80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=4 SACK_PERM=1
82.195.132.92	192.168.0.220	TCP	66	80→5057 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1380 WS=0 SACK_PERM=1
192.168.0.220	82.195.132.92	TCP	54	5057→80 [ACK] Seq=1 Ack=1 Win=262140 Len=0

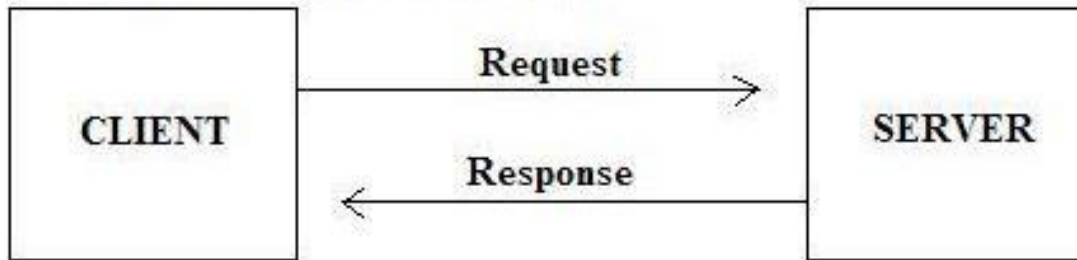
captura con el software wireshark



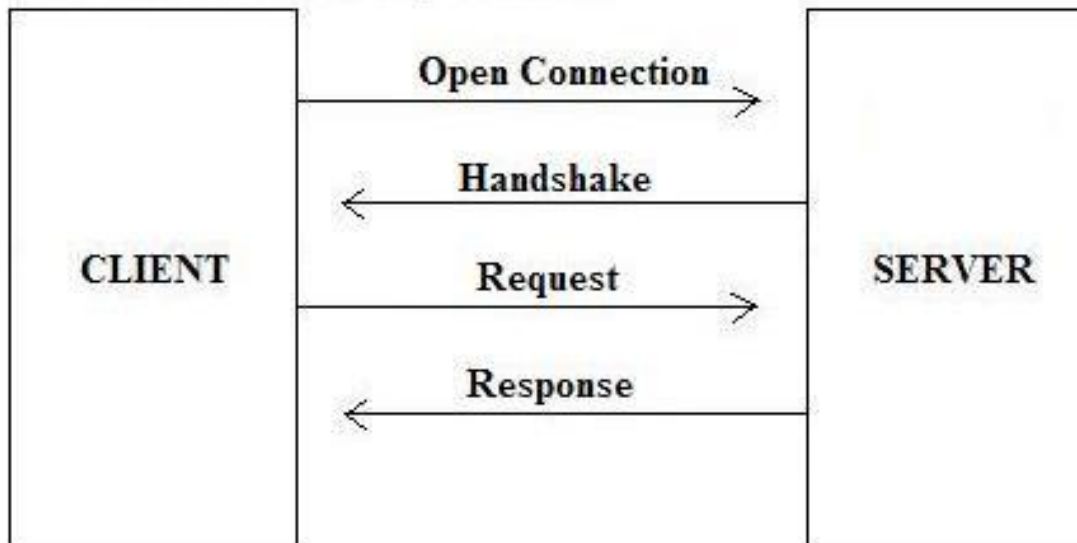
Establecimiento de una conexión

(TCP vs UDP)

UDP Request / Response Paradigm

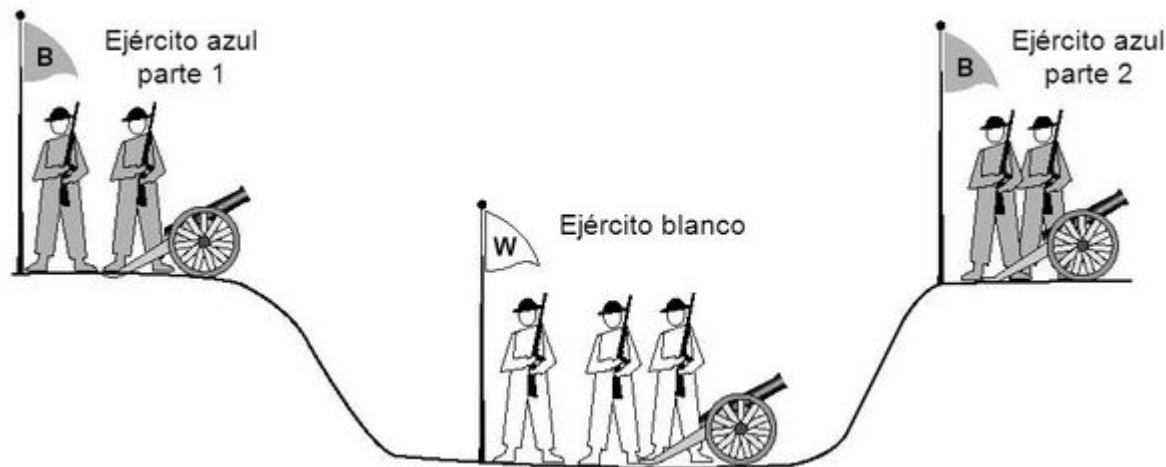


TCP Handshake Paradigm



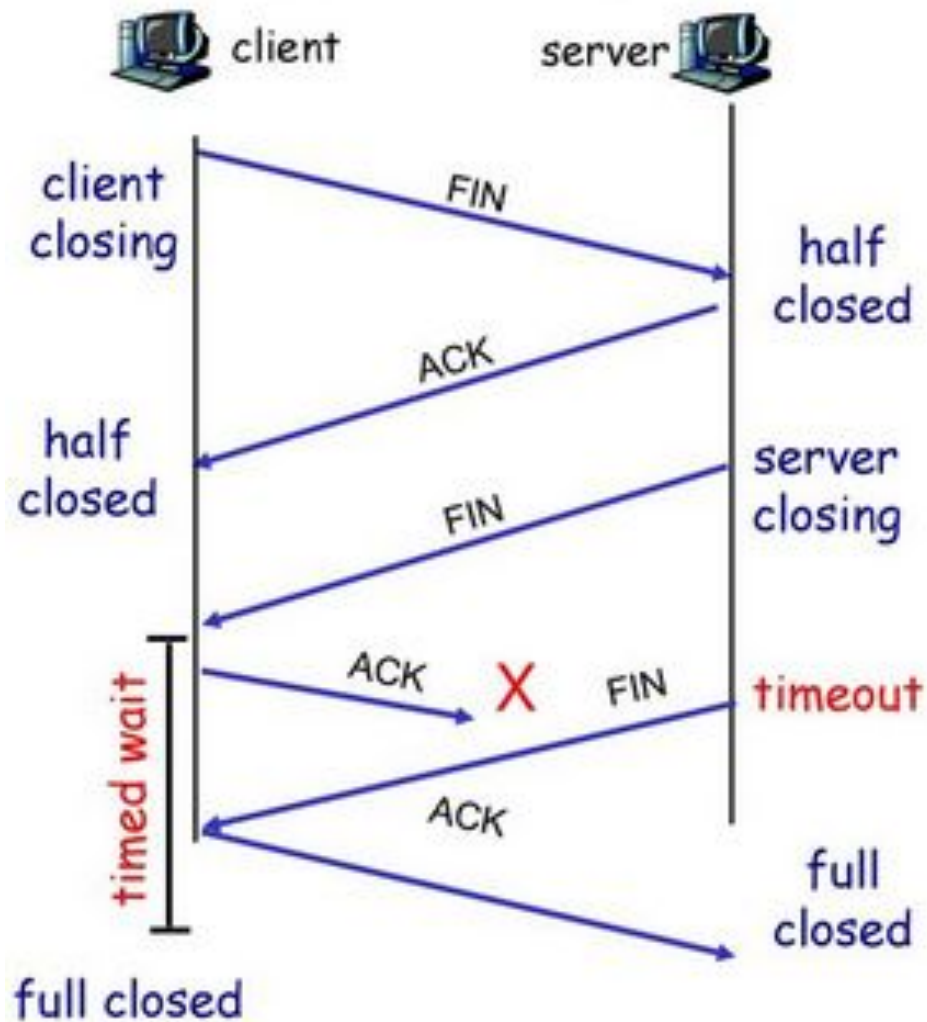
Cómo cerrar una conexión

Si ninguna de las partes está preparada para desconectarse hasta estar convencida de que la otra está preparada para desconectarse también, nunca ocurrirá la desconexión.



Problema intrínseco: Los dos ejércitos/ Los generales Bizantinos

Cómo cerrar una conexión FIN

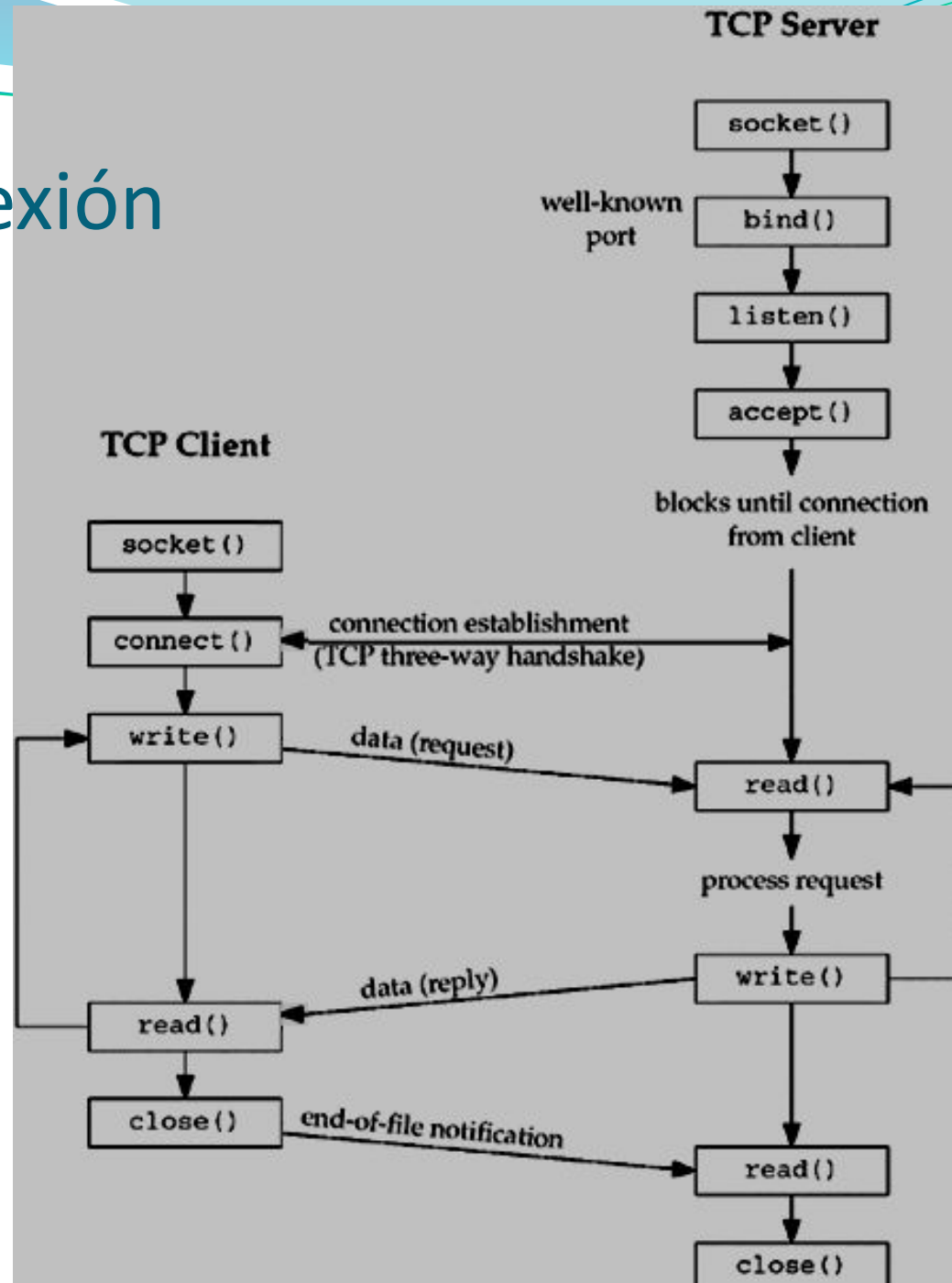


Cómo cerrar una conexión FIN

Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	TCP	66	22000 → 41162 [FIN, ACK] Seq=51 Ack=36 Win=43776 Len=0 TSval=1440312237
127.0.0.1	127.0.0.1	TCP	66	41162 → 22000 [FIN, ACK] Seq=36 Ack=51 Win=43776 Len=0 TSval=1440312237
127.0.0.1	127.0.0.1	TCP	66	22000 → 41162 [ACK] Seq=52 Ack=37 Win=43776 Len=0 TSval=1440312237 TSecr=
127.0.0.1	127.0.0.1	TCP	66	41162 → 22000 [ACK] Seq=37 Ack=52 Win=43776 Len=0 TSval=1440312237 TSecr=

Establecimiento y cierre de una conexión

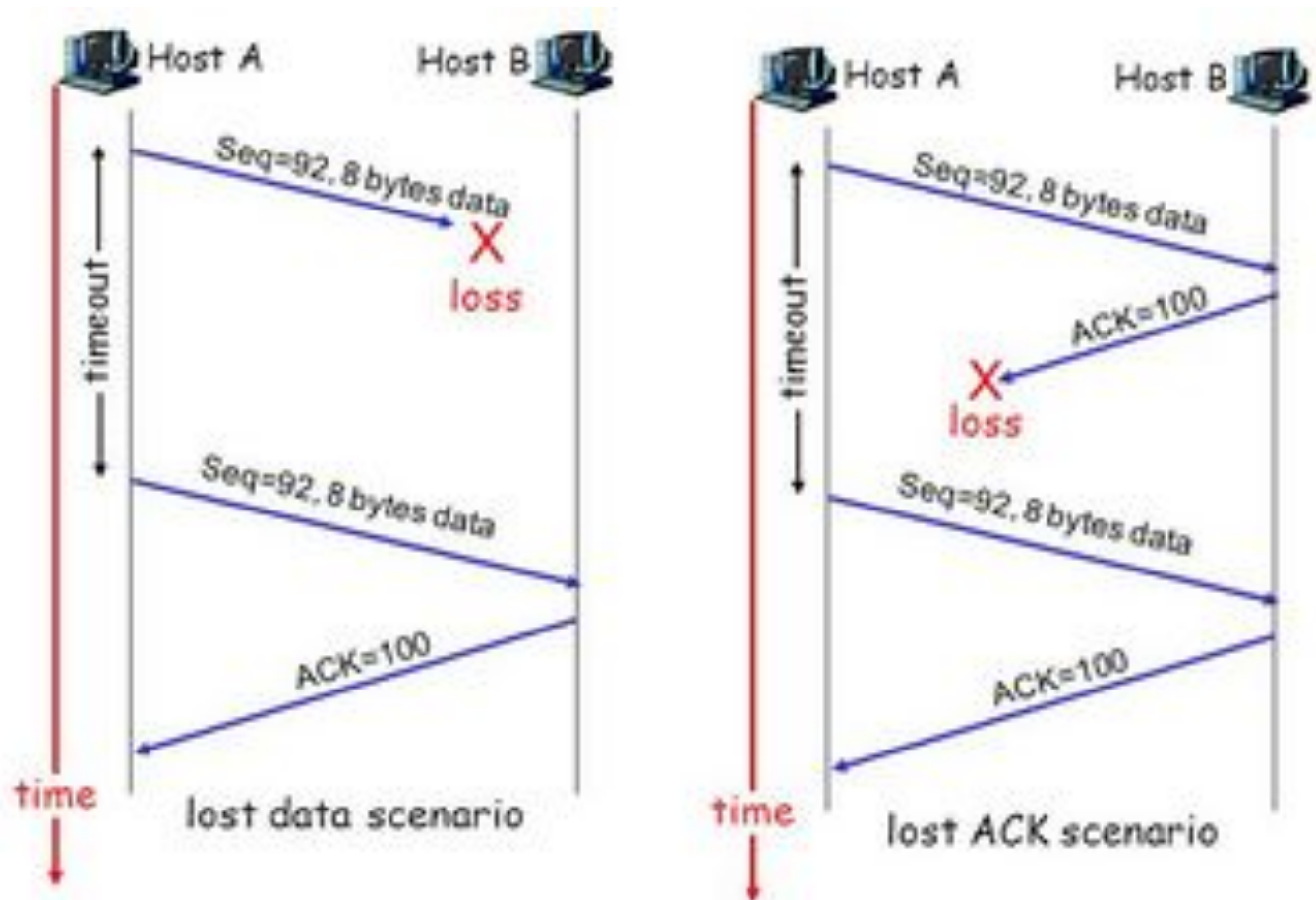
Ejemplo
detallado con sockets



Protocolo de Transporte de Internet: TCP

pérdida de envíos y acks

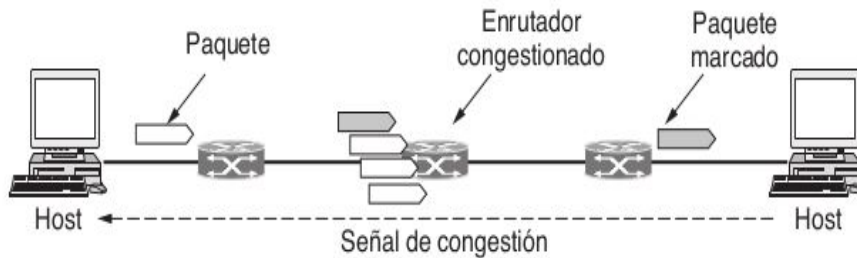
El protocolo TCP considera posibles escenarios de retransmisión:



En caso de no recibir respuesta se usa un timeout

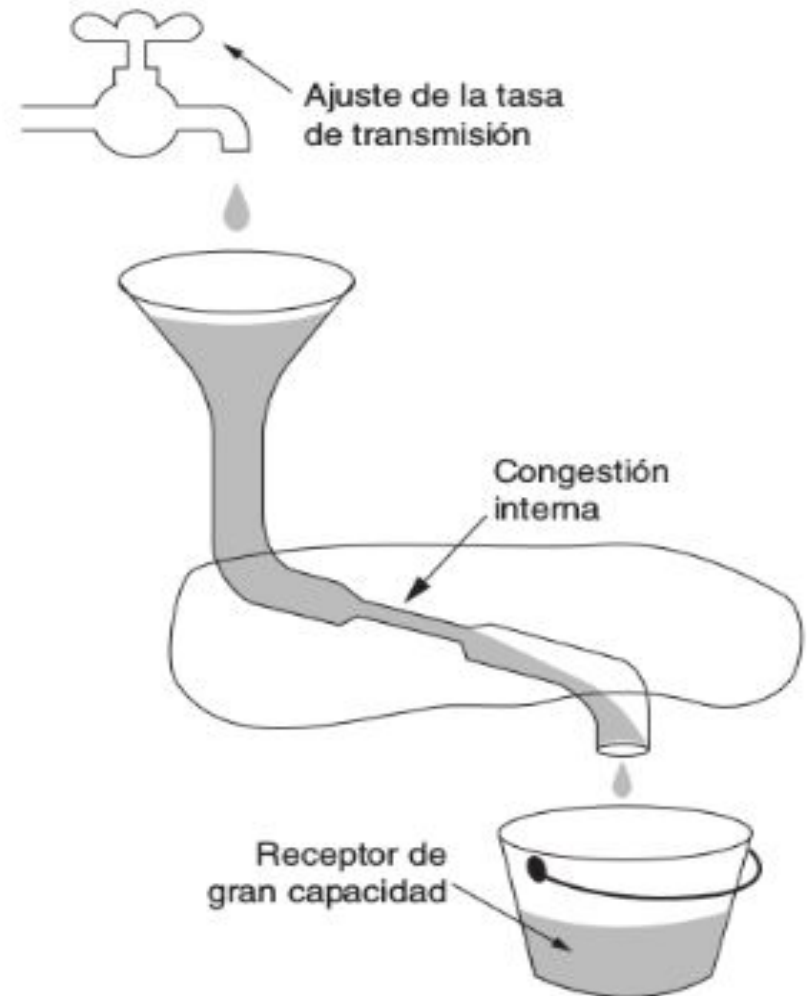
Control de Congestión

otra vez!



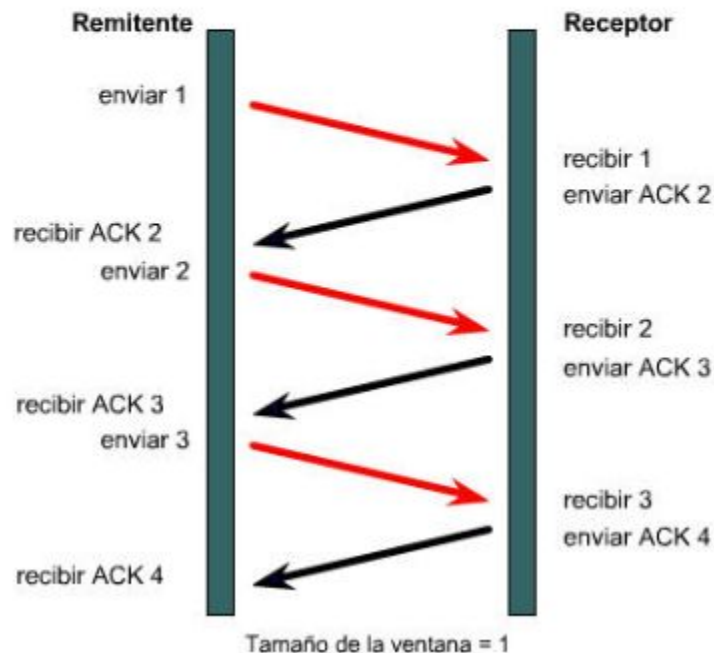
La capa de red proporciona información de que hay congestión en la red.

La capa de transporte reacciona regulando la tasa de transmisión como una canilla.



Control de Congestión

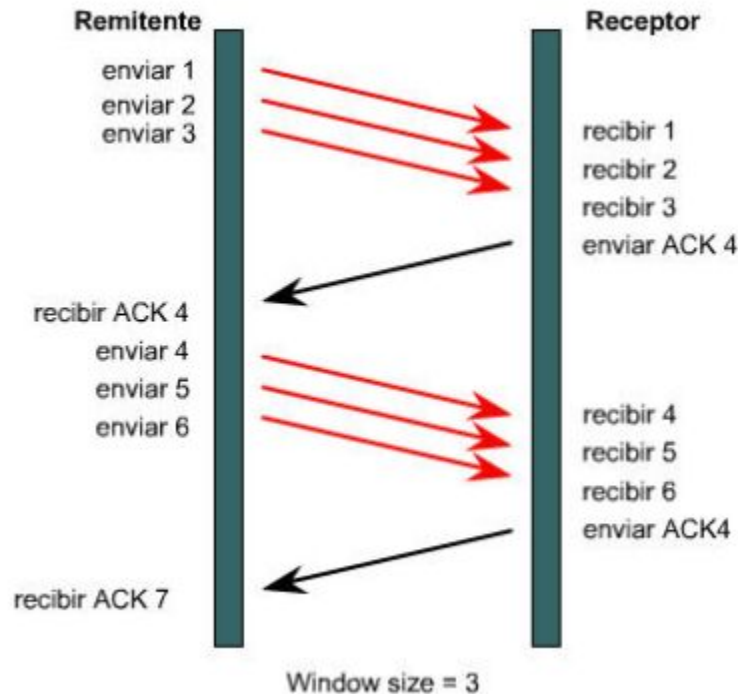
El tamaño de ventana determina la cantidad de datos que se pueden enviar simultáneamente antes que el destino responda con ACK



Después que un host transmita el tamaño de ventana en bytes, el host debe recibir un ack indicando que la información se recibió antes de poder enviar mas información.

Por ej.: ventana de 1, se debe generar ack por cada byte antes de enviar el siguiente.

Control de Congestión

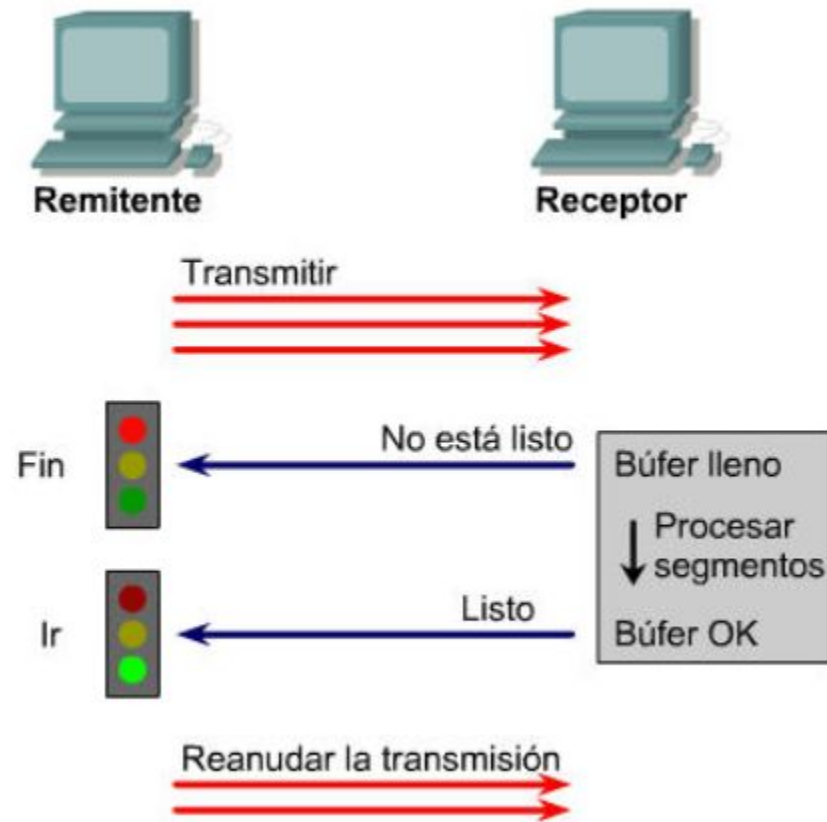


Los hosts negocian el tamaño de ventana antes del ack.

Tamaño de ventana TCP=3
Se determina de forma dinámica el tamaño.

La variación dinámica del tamaño de ventana incrementa la confiabilidad

Control de Congestión



Bibliografía

- Redes de Computadoras (5ta Edición)
Andrew Tanenbaum , Capítulo 6